Content-Based Satellite Image Retrieval for Earth Observation

Mohammad Abu Talha

Enrolment: D03000203

University of Naples Federico II

Course of Information Retrieval, Prof. Antonio Maria Rinaldi

A.Y. 2025/2026

Abstract

This report presents a content-based image retrieval (CBIR) system for satellite scenes, built and evaluated on the UCMerced Land Use dataset (Kaggle version, 21 classes, 10,500 images). Two pretrained models, ResNet50 and CLIP ViT-B/32, are used as feature extractors without fine-tuning, and their descriptors are fused by weighted concatenation. Four FAISS indexes (exact, HNSW, IVF-PQ and binary hash codes) are compared in terms of retrieval quality, agreement with exact search and memory per vector. On 1,050 queries against 9,450 database images, the fused model reaches P@1 = 0.998, P@10 = 0.931 and mAP = 0.575, above both single models. HNSW reproduces exact search almost perfectly, while IVF-PQ and binary hash codes reduce memory by about 160 and 32 times for a drop of 0.020 and 0.044 in P@10. The results come from a single random split of a dataset version that differs from the standard benchmark.

Chapter 1: Introduction to Information Retrieval in Earth Observation

Content-Based Satellite Image Retrieval on UCMerced Land Use. Information Retrieval course, A.Y. 2025/2026.

1.1 Context

Satellite and aerial sensors now produce Earth observation data at a very large scale. Archives grow every day and cover land use, vegetation, urban areas, water bodies and natural hazards. Artificial Intelligence has become a central tool for processing this data and extracting timely information from it [1]. However, analysing images is only useful if the right images can first be found. As archives reach millions of images, manual inspection or searching by file name or date is no longer practical.

1.2 Information Retrieval for satellite imagery

Information Retrieval (IR) addresses this problem by organizing a collection so that items relevant to a user's need can be located quickly. In Content-Based Image Retrieval (CBIR), the query is an image, and the system returns the images in a database that are most similar in content. For Earth observation, similarity usually means a similar land cover or scene type, such as agricultural fields, harbors, or residential areas.

CBIR for remote sensing faces specific challenges:

- **Semantic gap:** low-level pixel similarity does not always match what a user considers similar.
- **Visually close classes:** scenes such as dense, medium and sparse residential areas differ only in the density of buildings.
- **Scalability:** exact comparison against every image becomes costly in large archives, so compact and approximate index structures are needed [6].
- **Choice of representation:** deep embeddings from models pretrained on natural images and on image-text data [7, 8] can be used, but it is not obvious in advance which representation suits aerial scenes best.

1.3 Problem statement

The goal of this project is to build and evaluate a CBIR system for satellite scenes. Given a query image, the system must return a ranked list of database images, and a result is considered relevant if it belongs to the same land-use class as the query. The system should be accurate and also suitable for scaling to larger archives.

1.4 Objectives

1. Build a retrieval pipeline on the UCMerced Land Use dataset (Kaggle version, 21 classes, 10,500 images), after checking it for near-duplicate images.
2. Compare two pretrained feature extractors, ResNet50 and CLIP ViT-B/32, without fine-tuning.
3. Test whether fusing the two embeddings improves retrieval.
4. Evaluate scalable indexing options (exact search, HNSW, IVF-PQ and binary hash codes) in terms of retrieval quality, agreement with exact search, and memory per vector.
5. Evaluate all methods with the same split and metrics (precision@k and mean average precision) and analyse the results per class.

1.5 Organization of the report

Chapter 2 describes the data and preprocessing. Chapter 3 presents the retrieval methodologies. Chapter 4 describes the implementation and reports the evaluation. Chapter 5 summarizes the conclusions and possible future work.

Chapter 2: Data and Preprocessing

2.1 Data source

The experiments use the UCMerced Land Use dataset, a standard benchmark for CBIR in remote sensing. The original dataset contains aerial optical images collected from the USGS National Map Urban Area Imagery at 0.3 m/pixel resolution, grouped in 21 land-use classes with 100 images each (2,100 images in total).

The copy used here was downloaded from Kaggle (*landuse-scene-classification*) after the original server could not be reached. This version differs from the standard benchmark: it contains **500 images per class, 10,500 in total**, stored as PNG files in one folder per class. The reason for the larger size is not documented in the files we used, so it is stated here as a property of the data, not explained. Only the

folder `images/` was used. A second folder with a ready-made train/test/validation split was ignored, so that the split could be controlled and described exactly.

As a consequence, the results in this report are **not directly comparable** with published results on the standard 2,100-image benchmark.

The 21 classes are: agricultural, airplane, baseballdiamond, beach, buildings, chaparral, denseresidential, forest, freeway, golfcourse, harbor, intersection, mediumresidential, mobilehomepark, overpass, parkinglot, river, runway, sparseresidential, storagetanks and tenniscourt.

2.2 Image properties

The images are optical RGB scenes, so no spectral bands beyond red, green and blue are available. Each image shows one scene and carries one class label, which acts as the ground truth for retrieval. Because the data are RGB, the pretrained models used in this project can be applied without changing their input layers. Multispectral or SAR data would need different handling and are left for future work.

2.3 Data quality check

Since the dataset is larger than the standard one, it was checked for near-duplicate images, for example augmented copies of the same scene, which could make retrieval look better than it is if a query and its copy landed on opposite sides of the split.

The check used the ResNet50 embeddings described in Section 2.4. For each image, the cosine similarity to its nearest neighbour in the whole collection was computed. The counts are summarized below.

Nearest-neighbour similarity	Images
above 0.99	2
above 0.97	20
above 0.95	122

Only 0.2% of the 10,500 images have a neighbour above 0.97. This does not indicate widespread duplication, and similar-looking neighbours at lower similarity are expected in a land-use dataset (for example two tennis courts). These images were **kept**, and no image was removed. Their effect on the metrics is expected to be small but was not measured, and this is listed as a limitation in Chapter 5.

2.4 Preprocessing and feature extraction

Two pretrained models were used as fixed feature extractors, without fine-tuning.

- **ResNet50** (torchvision, ImageNet weights `IMAGENET1K_V2`). Images are converted to RGB, resized to 224×224 and normalized with the ImageNet mean and standard deviation. The classification layer is removed, and the 2048-dimensional output of the last pooling layer is used as the descriptor.
- **CLIP ViT-B/32** (OpenCLIP, weights `laion2b_s34b_b79k`). Images are preprocessed with the transform that ships with the model, and the 512-dimensional image embedding is used as the descriptor.

In both cases every descriptor is **L2-normalized**, so that the inner product between two descriptors equals their cosine similarity. This lets the same similarity measure be used by all indexes in Chapter 4. Embeddings were computed once on a GPU and stored on disk, so that every experiment reuses exactly the same features.

2.5 Database and query split

The collection was split once, at random, **within each class** (fixed seed 42):

Part	Per class	Total
Queries	50	1,050
Database	450	9,450

A query is never part of the database, and each query has 450 relevant images in the database (the images of its own class). The same split is used for every method, so differences between methods are not caused by different data. Only one split was evaluated.

Chapter 3: Retrieval Methodologies

3.1 Overview of retrieval approaches in Earth observation

Several retrieval formulations exist for Earth observation: content-based retrieval, where the query is an image; change-detection retrieval over multi-temporal image pairs; multi-modal retrieval, where text queries are matched to images with vision-language models such as RemoteCLIP and GeoRSCLIP [7, 8]; object-based retrieval of images containing specific features; and event-based retrieval of extreme events. Each answers a different user need, and each needs different data (time series, image-text pairs or object annotations).

This project focuses on **content-based image retrieval (CBIR)**. It is the most direct formulation, it needs only images with class labels, and it makes it possible to study the two aspects that matter most for large archives: the quality of the image representation and the scalability of the index.

3.2 General pipeline

The system follows the classical CBIR scheme in four stages:

1. **Feature extraction:** each image is converted into a fixed-length vector (descriptor) by a pretrained network.
2. **Indexing:** the descriptors of the database images are stored in a search structure.
3. **Search:** the query image goes through the same feature extractor, and the index returns the database images ranked by similarity.
4. **Evaluation:** a result is relevant if it has the same land-use class as the query.

All descriptors are L2-normalized, so similarity is measured with the cosine similarity, computed as an inner product. Higher means more similar.

3.3 Image representations

ResNet50. A convolutional network trained for image classification on ImageNet. With its classification layer removed, it gives a 2048-dimensional descriptor. Convolutional features are sensitive to texture and local structure, which is useful for scenes such as roads, tanks and bridges.

CLIP ViT-B/32. A vision transformer trained on large collections of image-text pairs, which gives a 512-dimensional descriptor. Because it is trained with language supervision, its embeddings tend to reflect what objects and scenes are shown. Only the image encoder is used here. The text encoder is not needed for CBIR, but would allow text queries in an extension, as in [7, 8].

Neither model was fine-tuned on remote sensing data, so both are used as general-purpose feature extractors.

3.4 Fusion of the two representations

The two models were expected to make different errors, so their descriptors were combined by **weighted concatenation**. With normalized ResNet50 descriptor a and normalized CLIP descriptor b, and weight w for ResNet50, the fused descriptor is

$$[\sqrt{w} \cdot a \,,\ \sqrt{(1 - w)} \cdot b]$$

which has 2560 dimensions and unit length. The inner product of two fused descriptors equals w times the ResNet50 similarity plus $(1 - w)$ times the CLIP similarity, so fusion is a weighted average of the two cosine similarities, computed in a single index. The main configuration uses the simple average, $w = 0.5$. Values of w from 0.3 to 0.7 were tested as a sensitivity analysis.

3.5 Indexing methods

Four indexes from the FAISS library were compared on the fused descriptors. Exact search is the reference, and the other three trade accuracy for speed or memory, as needed in large archives [6].

Index	Type	Main parameters	Memory per vector
Flat	Exact inner-product search	none	10,240 bytes (2560 × 4)
HNSW	Approximate, graph-based	32 neighbours per node, efSearch 256	about 10,240 bytes plus the graph
IVF-PQ	Approximate, inverted file with product quantization	64 cells, 16 probed, 64 sub-quantizers of 8 bits	64 bytes (compressed code)
Binary hash	Hash codes compared with Hamming distance	1 bit per dimension	320 bytes (2560 bits)

- **Flat** compares the query with every database vector. It is always correct and serves as ground truth for the other indexes.
- **HNSW** builds a multi-layer proximity graph and searches it greedily, so it visits only a small part of the database. Vectors are stored in full, so memory is not reduced.

- **IVF-PQ** divides the space into cells and only searches the closest ones. Vectors are compressed with product quantization, which greatly reduces memory but makes distances approximate.
- **Binary hash** turns each descriptor into a bit string by taking the sign of each dimension after subtracting the database mean. Hamming distance on bit strings is very fast and compact. This is a simple hashing baseline, not a learned hash code such as the methods in [6], which are listed as future work.

3.6 Evaluation measures

- **Precision@k (P@k):** the fraction of the top-k results that belong to the query's class, averaged over all queries. P@1 asks whether the best result is correct.
- **Mean average precision (mAP):** the average, over queries, of the precision measured at each relevant result in the ranking, divided by the number of relevant images (450 per query). For the exact models the full ranking of 9,450 images is used. For the index comparison, mAP is computed on the top 1,000 results (mAP@1000) so that all indexes are measured in the same way, and it is not comparable with the full-ranking mAP.
- **Top-10 overlap and recall@100 against exact search:** the share of the exact top-10 or top-100 results that an approximate index also returns. They measure agreement with exact search, which is a different thing from retrieval quality measured with class labels.
- **Memory per vector**, in bytes.

Query times were recorded but vary strongly between runs at this database size, so they are not used for conclusions.

Chapter 4: System Implementation and Evaluation

4.1 Implementation and environment

The system was implemented in Python in a Google Colab notebook. Feature extraction ran on a GPU, and indexing and search ran on the CPU with FAISS.

Item	Value
Python	3.13.16
GPU	Tesla T4 (feature extraction only)
torch / torchvision	2.11.0 / 0.26.0
open_clip_torch	3.3.0
faiss-cpu	1.15.1
numpy / pandas	2.4.6 / 2.2.3
pillow / matplotlib	11.3.0 / 3.10.0

The components are: a dataset loader (images and labels from the class folders), two feature extractors (ResNet50 and CLIP), a fusion step, a FAISS index (four variants), and an evaluation module that

computes the metrics of Section 3.6 from the ranked lists. Embeddings are saved to disk after extraction, so indexing and evaluation can be repeated without recomputing them. The complete code, the list of libraries and instructions are delivered with this report.

4.2 Evaluation protocol

- **Queries and database:** 1,050 queries (50 per class) and 9,450 database images (450 per class), from the single split with seed 42 described in Section 2.5.
- **Relevance:** a database image is relevant if it has the same class as the query. Each query has 450 relevant images.
- **Ranking:** for the model comparison, the whole database is ranked, and mAP uses the full ranking. For the index comparison, mAP is computed on the top 1,000 results.
- **A correction made during the work:** a first version of the index comparison computed average precision by dividing by the number of relevant images found in the returned list, and counted empty slots returned by approximate indexes as if they were labels. This inflated mAP for the approximate indexes. The values reported here divide by the true number of relevant images (450), and ignore empty slots.

4.3 Results: comparison of representations

Model	Dim	P@1	P@5	P@10	P@20	P@50	mAP
ResNet50	2048	0.9914	0.9417	0.8872	0.8391	0.7651	0.5046
CLIP ViT-B/32	512	0.9733	0.9299	0.8939	0.8532	0.7919	0.5283
Fused (w = 0.5)	2560	0.9981	0.9657	0.9310	0.8906	0.8244	0.5748

ResNet50 is better than CLIP at the top of the ranking (P@1 and P@5), and CLIP is better deeper in the ranking (P@20, P@50 and mAP). The gap in P@10 between the two (0.007) is small. The fused model is best on every metric: compared with CLIP, mAP rises by 0.046 and P@10 by 0.037, and compared with ResNet50, mAP rises by 0.070.

The mAP values look low next to precision@k. This is expected: with 450 relevant images per query in a database of 9,450, any relevant image ranked late lowers mAP.

Per-class results

Class	ResNet50	CLIP	Fused
agricultural	0.990	0.998	0.992
airplane	0.988	0.994	0.988
baseballdiamond	0.876	0.942	0.952
beach	0.982	0.994	0.992
buildings	0.722	0.800	0.840

Class	ResNet50	CLIP	Fused
chaparral	1.000	0.990	1.000
denseresidential	0.698	0.710	0.774
forest	0.986	0.976	0.994
freeway	0.948	0.774	0.966
golfcourse	0.848	0.876	0.898
harbor	1.000	0.992	1.000
intersection	0.802	0.888	0.886
mediumresidential	0.790	0.778	0.862
mobilehomepark	0.832	0.864	0.902
overpass	0.840	0.738	0.852
parkinglot	0.998	0.996	1.000
river	0.850	0.890	0.926
runway	0.982	0.946	0.994
sparseresidential	0.856	0.896	0.926
storagetanks	0.830	0.764	0.866
tenniscourt	0.814	0.966	0.942

Per-class P@10. The two models are strong on different classes. CLIP is clearly better on tenniscourt (+0.152), intersection (+0.086), buildings (+0.078) and baseballdiamond (+0.066). ResNet50 is clearly better on freeway (+0.174), overpass (+0.102) and storagetanks (+0.066). A possible explanation is that CLIP captures what objects and scenes are shown, while ResNet50 is more sensitive to texture and structure such as roads and tanks. This explanation was not tested. The fused model improves on the weaker of the two models on every class, and is above both on, for example, freeway, overpass, storagetanks and mediumresidential. On a few classes it is slightly below the best single model (for example tenniscourt, 0.942 against 0.966).

The hardest class remains **denseresidential** (0.774 with fusion), followed by **buildings** (0.840) and **mediumresidential** (0.862). These classes are visually close to each other, and differ mainly in building density.

Sensitivity to the fusion weight

w (ResNet50)	P@10	mAP
0.3	0.9311	0.5816

w (ResNet50)	P@10	mAP
0.4	0.9330	0.5811
0.5	0.9310	0.5748
0.6	0.9268	0.5643
0.7	0.9189	0.5511

Between w = 0.3 and w = 0.5 the results change little (about 0.007 in mAP), and they decrease when ResNet50 receives more weight. The weights were examined on the same queries used for evaluation, so the best values are slightly optimistic. For this reason, the untuned w = 0.5 is used as the main result.

4.4 Results: indexing methods

All indexes were built on the fused descriptors (w = 0.5).

Index	P@10	mAP@1000	Top-10 overlap with exact	Recall@100 vs exact	Bytes per vector
Flat (exact)	0.9310	0.5265	n/a	1.0000	10,240
HNSW	0.9310	0.5436	1.0000	0.9999	10,240
IVF-PQ	0.9106	0.5737	0.7018	0.7863	64
Binary hash	0.8868	0.4007	0.5309	0.4991	320

- **HNSW** returns practically the same results as exact search (top-10 overlap 1.00, recall@100 0.9999) with the same P@10, so at this scale it loses no quality. Since it stores full vectors, it does not save memory. Its benefit is search speed on much larger archives.
- **IVF-PQ** reduces memory about 160 times (10,240 to 64 bytes per vector). P@10 falls by 0.020. Its top-10 overlap with exact search is only 0.70, but P@10 changes little, which means that most of the results it swaps in still belong to the correct class.
- **Binary hash codes** reduce memory 32 times. P@10 falls by 0.044, and agreement with exact search is the lowest (recall@100 about 0.50).

Agreement with exact search and retrieval quality measured with class labels are different quantities. The binary index disagrees with exact search on about half of the top 100, but its P@10 is still 0.887.

The mAP@1000 of HNSW and IVF-PQ is higher than that of exact search. This should not be read as approximate search being better. Exact search ranks by feature similarity and not by class, so a class-based metric can favour an approximate index by chance. One possible explanation is that approximate indexes mostly return images from the neighbourhood of the query, which are more often from the same class, but this was not tested. mAP@1000 is therefore reported only as a secondary column.

Speed. Query times were measured, but with 9,450 images they vary between runs (for exact search from about 1.3 to 3.8 ms per query in repeated runs of the same configuration), so no speed conclusion is drawn. The advantages of HNSW and IVF-PQ in speed would have to be tested on a much larger archive.

4.5 Discussion

The results answer the objectives of Chapter 1 as follows. First, both pretrained models give a strong baseline without any training: P@1 is above 0.97 and P@10 close to 0.89. Second, they are complementary, and a simple fusion without tuning improves all metrics, with P@10 reaching 0.931. Third, scalable indexing is possible with a limited loss of quality: HNSW is practically lossless, and IVF-PQ and binary hash codes trade a few points of P@10 for much smaller indexes.

These conclusions are restricted by the conditions of the experiment: a single random split, a dataset version that differs from the standard benchmark, relevance defined only by the class label, and a database of 9,450 images.

Chapter 5: Conclusions and Future Work

5.1 Summary of the work

This project built and evaluated a content-based image retrieval system for satellite scenes on the UCMerced Land Use dataset (Kaggle version, 21 classes, 10,500 images). Two pretrained models, ResNet50 and CLIP ViT-B/32, were used as feature extractors without fine-tuning, their descriptors were fused, and four FAISS indexes (exact, HNSW, IVF-PQ and binary hash codes) were compared. Evaluation used 1,050 queries against a database of 9,450 images, with precision@k and mean average precision, and class labels as the definition of relevance.

5.2 Main findings

- **Pretrained models are a strong baseline.** Without any training, ResNet50 reaches P@1 = 0.991 and P@10 = 0.887, and CLIP reaches P@1 = 0.973 and P@10 = 0.894.
- **The two models are complementary.** ResNet50 is better at the top of the ranking and CLIP deeper in it, and each is clearly better on different classes (for example CLIP on tenniscourt, ResNet50 on freeway).
- **Fusion gives the best results.** The untuned average of the two (w = 0.5) improves every metric: P@1 = 0.998, P@10 = 0.931 and mAP = 0.575, against 0.528 for the best single model.
- **Scalable indexing is possible with limited loss.** HNSW returns practically the same results as exact search. IVF-PQ reduces memory about 160 times for a drop of 0.020 in P@10, and binary hash codes reduce it 32 times for a drop of 0.044.
- **Agreement with exact search is not the same as quality.** Indexes that disagree strongly with exact search (IVF-PQ, binary hash) still keep most of their precision against the class labels.
- **Residential classes remain difficult.** Dense, medium and sparse residential areas and buildings are the weakest classes for all methods.

5.3 Strengths and limitations

Strengths. The pipeline is simple, needs no training and no paid resources, and every method is evaluated with the same split and metrics. The results are accompanied by checks on the data (near-duplicates) and on the metric (a correction of the mAP computation, Section 4.2).

Limitations.

- **One random split only.** Small differences, such as the 0.007 gap in P@10 between ResNet50 and CLIP, are not statistically established. The conclusions about fusion rest on larger gaps, but were also measured on one split.
- **Fusion weight explored on the evaluation queries.** The best weights in the sensitivity analysis are therefore slightly optimistic, and for this reason the main result uses w = 0.5.
- **Dataset version.** The Kaggle copy has 500 images per class instead of the 100 of the standard benchmark.
- **Near-duplicates were kept.** The check found 20 images with a neighbour above 0.97 cosine similarity (2 above 0.99). They were not removed, and their effect on the metrics was not measured, although it is expected to be small.
- **Relevance by class label only.** Two images of the same class are not necessarily similar in content, and images of different classes can be similar (for example the residential classes). Precision measured with labels may underestimate the usefulness of the results.
- **Small scale.** With 9,450 database images, search is fast for every index, so no conclusion about speed could be drawn. The benefits of HNSW and IVF-PQ for time would have to be shown on a larger archive.
- **Simple hashing baseline.** The binary codes are the sign of each dimension and are not learned, so they do not represent what learned hash codes [6] can achieve.
- **Untested explanations.** The explanations given for the per-class differences and for the higher mAP@1000 of approximate indexes are hypotheses.

5.4 Future work

- **Better representations.** Use a model pretrained on remote sensing data, such as RemoteCLIP [8] or GeoRSCLIP [7], or fine-tune ResNet50 or CLIP on the database with a metric-learning loss (triplet or Siamese networks).
- **Learned hash codes.** Compare the simple binary codes with learned methods such as asymmetric hash code learning [6].
- **More reliable evaluation.** Repeat the experiments on several random splits and report mean and standard deviation, choose the fusion weight on separate validation queries, and repeat the analysis on the standard 2,100-image UCMerced benchmark for comparison with published results.
- **Duplicate handling.** Measure the effect of removing the near-duplicate images.
- **Relevance feedback.** Add active learning with relevance feedback, so that users can refine results interactively.
- **Larger archives.** Test the indexes on much bigger collections, where differences in speed and memory become important.
- **Other data and tasks.** Extend to multispectral or SAR imagery, and to text queries with vision-language models, which would lead to multi-modal retrieval [7, 8, 9]. A study of this kind, with several datasets and rigorous evaluation, could be developed into a research thesis.

References

[1] Aidan Duggan, Bruno Andrade, and Haithem Afli. "Advancing Earth Observation: A Survey on AI-Powered Image Processing in Satellites". arXiv preprint arXiv:2501.12030, 2025.

[2] Gabriele Meoni et al. "Unlocking the use of raw multispectral earth observation imagery for onboard artificial intelligence". IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing, 2024.

[3] Ivica Dimitrovski et al. "Aitlas: Artificial intelligence toolbox for earth observation". Remote Sensing 15(9), 2023, p. 2343.

[4] Lynn Miller, Charlotte Pelletier, and Geoffrey I. Webb. "Deep Learning for Satellite Image Time-Series Analysis: A review". IEEE Geoscience and Remote Sensing Magazine, 2024.

[5] Zhitong Xiong et al. "Earthnets: Empowering AI in earth observation". arXiv preprint arXiv:2210.04936, 2022.

[6] Weiwei Song et al. "Asymmetric Hash Code Learning for Remote Sensing Image Retrieval". IEEE Transactions on Geoscience and Remote Sensing 60, 2022, pp. 1-14.

[7] Zilun Zhang et al. "RS5M and GeoRSCLIP: A Large Scale Vision-Language Dataset and A Vision-Language Foundation Model for Remote Sensing". arXiv preprint arXiv:2306.11300, 2023.

[8] Fan Liu et al. "RemoteCLIP: A Vision Language Foundation Model for Remote Sensing". IEEE Transactions on Geoscience and Remote Sensing 62, 2024, pp. 1-16.

[9] Kartik Kuckreja et al. "GeoChat: Grounded Large Vision-Language Model for Remote Sensing". Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024, pp. 27831-27840.

[10] Yi Yang and Shawn Newsam. "Bag-of-visual-words and spatial extensions for land-use classification". Proceedings of the 18th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems, 2010.